LLM-Fleet Blueprint — the Cockpit layer (planned: Session 44, 2026-10-29)

Status: **PLAN** — nothing here is built yet; every adoption is gated by its named wave (W0c / W4 / W5) and the verification rules in `FULL-SYSTEM-PLAN.md`. Where a claim came from outside this workspace it is labeled AUTHORITATIVE-TECHNICAL, VENDOR, or PRIMARY-AUTHOR; vendor and author claims are never treated as fact.

Companion documents: `FULL-SYSTEM-PLAN.md` (R1–R8, 10 stages) · `AGENT-FLEET-BLUEPRINT.md` (the ~75 deterministic trading cells, R3 rule, §8.6 memory, L0–L4 kill cascade) · `ENGINE-DATABASE-PLAN.md` (D0–D7 stores; §19 = D1 decision).

---

§1 · What this layer is

The trading cells (≈75 owner-locked instruments; owner sometimes says "73" — the blueprint roster of 75 is the source of truth) are **deterministic**: rules, ONNX/frozen formulas, no LLM in the order path (R3 non-negotiable). This blueprint is everything around them that **is** LLM: a small fleet of agent workers + a chat plane the owner talks to. Its jobs:

Job	Who it serves	Hard rule
Build, review, test the fleet's code	owner	LLM edits code, gates verify
Watch data health (feeds, gaps, calendars)	E1/R1	findings → tickets, never silent fixes in prod
Security / secret / egress audits	owner	audit-only, no prod creds
News & event digests on R3	Mode R research	gated by E4-style replay tests before any score is trusted
Analysis/scoring on R3 (fundamentals, sentiment)	D5 score pipeline	concrete scores-versioned, edge-lift (E2) gate before trust
Ops: runbooks, incident response, restart drills	fleet ops	proposes actions; executes only via approval-ladder commands
Fleet control: watch ~75 cells, surface state, queue promotions	owner cockpit	read-mostly; control commands follow L0–L4
Training / self-improvement loop	§8 of fleet blueprint	offline only
Cockpit chat	owner	never an execution authority on its own

Never: the LLM layer does not place, size, or stop orders; it does not touch R1/R2 runtime; it cannot run long without its instrument; every conversation and action is logged to D4 (audit).

§2 · Control-plane roster (planned: 12 domains, headless)

Small by design — one worker per domain, spawned by the orchestrator, each with an explicit success condition and exit. Roles are *targets for W0c*; the mechanism can be any of the harnesses already wired in this workspace (`agency-agents` roster, day-trader skill pack), the ones we build.

#	Domain	Breeds from (adopted source)	Output artifact	Spawns when
1	**build**	agency-agents (build/architect bodies) + engine docs	PRs + test evidence	W0–W5 engineering
2	**review**	agency-agents code-review bodies	review comments + blocker list	every PR
3	**cybersec**	agency-agents security bodies + §12 audit loop	audit findings + fix PRs	weekly + pre-release
4	**data-QA**	this plan §12 wet-runs	gap calendar + anomaly tickets	nightly + on rail alert
5	**news**	TradingAgents role patterns (analyst digesters)	dated digests + citation card	market hours + scheduled
6	**research/analysis**	quant-mind artifact schema + agent retrieval; timecopilot beat-the-baseline forecast gate; Nixtla's free `statsforecast/mlforecast` (CPU) if forecast tooling needed	cited claims (chunk+page+quote+as_of) → D5	scheduled + on request
7	**scoring**	day-trader-skill procedure + laya-mlx decision-head contract (typed choice/score/noul)	score versions + veto calls	per research batch
8	**ops**	runbooks + brigade-style tasking	incident narratives + recovery runs	rail/cell alerts
9	**fleet-control**	brigade orchestration mechanics (below)	cell state map + rehearsal queue	cockpit session + scheduled
10	**training**	§8 fleet plan; laya decision-head contract (via upstream torch on Colab) + laya-style calibration clamps	new runs + E4 apps	scheduled (offline)
11	**compliance**	§7 R9–R14 + O11/O13 checklists	filing/blocker list	weekly
12	**cockpit/comms**	this §8	chat, reports, owner decisions recorded to vault	always-on

Deliberately **not** roles: "alpha generator on demand", "market caller" — anything whose output is a trade intent. Trade intent can only originate from the deterministic E2/E5 path; analysis agents may only *refine frozen artifacts* that themselves pass E4 again.

§3 · Orchestration — how agents are run, coordinated, and kept cheap

Chosen pattern (dossiers A/B/C/D read and integrated; architecture fixed, exact libraries named with evidence in §3.1):

Element	Plan	Adopted from	Note
Boss loop core	DAG task orchestrator: our `llm_tasks` Postgres table as the **durable** queue (dossier A/B verdict: ruflo's bus is in-proc only, brigade's is WS-only — durable queueing is ours regardless), with the ruflo `@claude-flow/swarm` orchestrator contract (priorities, retries, cycle-detection, worker release) ported as the runner library	ruflo swarm + our Postgres	"four imports, not three rewrites" (dossier A)
Loop discipline	brigade `loop-runner` / `loop-guards` / `slop-detector` ported as the turn-governor: budgets, no-progress fingerprint, repetition guard, independent done-checks, completion-gate veto	brigade (vendor-pinned fork)	"stops only when evaluateDone's checks pass, never on the agent's own say-so" — exactly our guard spirit
Approval ladder	request/resolve **contract** from brigade `approval-bridge.ts`, mechanism = ours on NATS subjects + cockpit WS	brigade pattern	L0–L3 mapping unchanged
Scheduled seats	brigade `cron/isolated-agent/run-executor.ts` + metering pattern, or our systemd timers if the brigade runtime is rejected in O16	brigade pattern	metering feeds G2/G9
Personas	agency-agents finance/security/testing personas + runbooks.json slug-roster contract (CI-checked)	agency-agents	sanitize marketing prose before user-facing use
Memory substrate	R3 semantic memory: brigade Tideline patterns (hybrid recall, decay, origin scoping, **write-gate**) vs ruflo `@claude-flow/memory` (HNSW hybrid + consolidator + FTS5 fallback) — decide at W0c by measured recall/latency toy-shootout, code-graph recall handled by Graphify (§8.1)	brigade / ruflo	write-gate + origin-scoping map 1:1 to our §8.2 write authority
Transport/comms	NATS JetStream subjects per seat + per event type (existing D2 rail)	ours	same bus the cells use = one ops path
Artifacts	scores/prompts/citations versioned in D5; code in git PRs	D5 exists	nothing of value lives only in chat
Approval enforcement	L0 (read-only) auto · L1 (sandbox) auto within caps · L2 (prod config change) owner + guardian sign-off · L3 (deploy/live change) owner only, logged to vault	L0–L4 cascade already exists	LLM never self-approves
Budget caps	per-task token cap, per-domain daily cap, hard off-switch in cockpit; per-run metering	—	trips → agent park + owner ping
Secrets	vault (no prompts contain keys); egress allow-list per domain (cybersec: none; news: feeds only)	§12	enforce at orchestrator level

§3.1 Deep line-by-line: the 13 owner-named repos → one verdict table

Table = **15 rows**: row 0a = brigade (owner favorite, pulled out as the orchestrator discussion), row 0b = Graphify (owner's late addition, detailed in §8.1), rows 1–12 = the owner's remaining 12 repos in his listed order.

#	Repo	Verdict	What we take	Where it slots
0a	**spinabot/brigade** (TS, MIT, ★11,365, last commit 2026-09-15 = 44-day lull, read via partial clone @ `4c2a18f` after repeated clone failures)	ADOPT-PRIMITIVES (owner-favorite; NOT wholesale runtime)	best loop-discipline code of all 13: `src/agents/loop/{loop-runner,loop-guards}.ts` (budget maxIterations/maxMs/maxTokens, no-progress fingerprint, repetition guard, **independent done-checks — "stops only when evaluateDone's checks pass, never on the agent's own say-so"**, completion-gate veto) + `quality/slop-detector.ts` (bounded repair retries); `src/agents/approval-bridge.ts` request/resolve contract → our L0–L3 ladder; `cron/isolated-agent/run-executor.ts` + metering → scheduled seats; `src/agents/memory/tideline` (hybrid recall + decay + origin scoping + write-gate) → R3 semantic-memory candidate §8.2; gateway headless daemon = cockpit transport candidate. 571 test files. CAVEATS: org-chart "governs who talks to whom" is INERT stage-A config (README overclaim, `src/agents/org/types.ts`); WS-only transport (no NATS/Postgres adapters); Convex/pi-SDK/30-channel-adapter baggage; brand-scrubbed fork → **vendor-pinned fork only, never track upstream**	loop guards + approval + cron = §3 boss loop; Tideline patterns §8.2; gateway decision = O16
0b	**Graphify-Labs/graphify** (Python, Apache-2.0 verified from LICENSE+NOTICE, ★~137,981, last commit 2026-10-27, source-read in full)	ADOPT-AS-TOOL	local tree-sitter AST graph, 0 LLM calls in code mode, EXTRACTED/INFERRED edge audit trail, native opencode skill install (`install.py:215-218`), `--update` incremental; Cloud upsell REJECTED (VENDOR "80%" claim)	§8.1 memory layer + W0c fit-test
1	**Jakeschincariol/day-trader-skill** (Python, MIT, ★2,692, active; 315/315 tests pass locally — measured by deep-read subagent)	ADOPT (code + skills)	11 SKILL.md pack; `trader-timemachine/timemachine.py` (956 LOC seeded Monte-Carlo of session bar-ratios, break-even-aware `1/(1+R)` verdicts, historical replay), `risk/check.py`, `size.py`: deterministic, stdlib-only chain usable **pre-LLM** as veto layer; excellent anti-overclaim prompt craft	skills → cockpit skill pack; veto tools → fleet-side scoring aid
2	**TauricResearch/TradingAgents** (Python, Apache-2.0 **verified from LICENSE**, ★117,967)	ADOPT-PATTERNS	best **typed schemas** of the 13: Pydantic `SentimentReport` (6-tier band + 0–10 score + confidence) + None/coercion fighters (`agents/schemas.py`) → D5 candidates; per-analyst subgraph wrapper w/ tool-round caps + typed report output (`graph/setup.py:_analyst_graph`); settle→alpha→reflection memory pattern (`backtest.py`, `memory/reflection.py`) grafts onto our E2 edge-lift gate; honest backtest, 90 test files; ~10–18 LLM calls/decision (cost real, not hidden). Strip: `llm_clients` vendor clients, CLI/Docker, simulated-trading bits	seats 5–6 schemas + debate mechanics; reflection pattern → §11
3	**ruvnet/ruflo** (TS, MIT, ★80,006, 982 open issues, commit 2026-10-28, read ~shallow: local clone was removed mid-reclone; tree inventory + raw-HTTP reads)	ADOPT-CORE-CONTRACT (cherry-pick 3 packages, reject the rest)	the only one of the three orchestration repos with a **headless, unit-tested DAG boss-loop**: `@claude-flow/swarm` `task-orchestrator.ts` (createTask/batch, priority queue, `wouldCreateCycle` detection, retry ladder maxRetries:3, worker release, `executeParallel`) + `agent-registry`/`agent-pool` + `@claude-flow/mcp` server infra = transport for Claude Code/opencode; plugin `guard`/`liveGuard` config ("refuse live order unless confirm/paper flag") = our G1–G6 shape as config contract. CAVEATS: **event bus is in-proc only (no durable queue — NATS glue stays ours)**; 8,265 files of sprawl, v3-alpha vs 3.63.8 line drift, `agentdb.rvf` binary in-tree; **synthetic README metrics** (download-badge alt text admits a fabricated counter; "Flash Attention speedup" marketing) → trust `__tests__/` + source only; 36 CI workflows incl. CodeQL/supply-chain = history of supply-chain diligence, still pin-and-audit	boss-loop contract shape (§3); `@claude-flow/memory` (HNSW hybrid, consolidator, FTS5 fallback) as R3 semantic-memory substrate candidate §8.2; liveGuard contract above G1–G6

#	Repo	Verdict	What we take	Where it slots
4	**msitarzewski/agency-agents** (Shell/markdown personas, MIT, ★164,928, commit 2026-10-24, local read)	ADOPT (already wired)	**267 single-file persona prompts** across 31 divisions (prompt-only, zero runtime — "a library of inputs, not a harness"); finance/ (investment-researcher, financial-analyst, fpa-analyst, bookkeeper, tax) + security/ + testing/ = seed roster for seats 1–12; `strategy/runbooks.json` slug-based roster + activation-phase contract → our crew-config schema; CI `check-runbooks.sh` pattern → copy as crew-config lint. CAVEATS: ~15 KB/persona token weight + embedded marketing metrics ("flagged $86M", "18% CTR") = sanitize before user-facing use; its "OpenCode drops agents past 119" claim is false on our box (499 registered measured)	role library for seats 1–12 (268 bodies already wired in this workspace as `agency__*`)
5	**HKUDS/Vibe-Trading** (Python, MIT **verified via LICENSE fetch**, ★37,056, read via full tree JSON + targeted HTTP — sections marked shallow)	ADOPT-PATTERNS (prompt packs)	best prompt packs of the 13: `swarm/presets/*.yaml` — 30+ YAML-defined teams with DAG dependencies, per-worker tool+skill whitelists, hard anti-fabrication rules ("Data Citation Discipline: every number must trace to a tool call / ground truth / upstream context"), bull/bear → risk → PM structured report exchange; honest per-call token accounting (`llm_usage.py`); 1,014 test files; live-broker side is deterministic-first and ported never	§7 skill grammar for seats 5–7; anti-fabrication rules adopted verbatim; presets emit **prose not typed schemas** — local schema pass still required
6	**HKUDS/AI-Trader** (Python, **NO LICENSE FILE — verified by find; README shows an MIT badge pointing at a LICENSE that does not exist (README overclaim)**, ★23,021, stale since 2026-06-11)	**REJECT**	not even a research framework — it is a hosted leaderboard/copy-trading **platform** where external agents self-report trades ("Platform records your provided price, does not verify if market is open" — `skills/ai4trade/SKILL.md`); production data-quality churn visible in repo (price-repair scripts); Alpha-Vantage-paid default	none
7	**TimeCopilot/timecopilot** (Python, MIT verified, ★630, commit 2026-10-27, Pre-Alpha 0.0.40; HTTP read — shallow)	ADOPT-PATTERNS (3 specific mechanics)	(1) **forced beat-the-baseline loop**: LLM output validated against SeasonalNaive cross-validation, `ModelRetry` if not beaten (`agent.py` validate output) → same shape as our E2 edge-lift gate, implementable with or without the LLM; (2) all-local statistical model set (AutoARIMA/AutoETS/Theta/Croston/SeasonalNaive…; foundation models local-torch; TimeGPT only an adapter shim we skip); (3) **`FunctionModel` stub-LLM test pattern** — agent flows tested deterministically without any provider = exact template for our G5 LLM eval harness + replay determinism. Skip wholesale install (Pre-Alpha API churn; heavy torch deps)	seat 6 forecast-gate design + G5 eval-harness template; live Ollama/offline-LLM check at W2
8	**LLMQuant/quant-mind** (Python, MIT, ★3,119, stale Aug-2026)	ADOPT-PATTERNS	cited-artifact schema (chunk+page+verbatim quote+as_of, SHA-256 ids); BM25/agentic retrieval; progressive-disclosure `contexts/` docs; NOT a shipped corpus — processor, not content	seat 6 grounding + our research store
9	**Nixtla/nixtla** (Python, repo LICENSE = **Apache-2.0 for the SDK** with service terms split out; the *service* is proprietary — GitHub NOASSERTION reflects the mix; ★4,038, commit 2026-10-28, local+HTTP read)	REJECT-AS-TOOL (cloud-locked), small pattern notes	code-verified: `nixtla/nixtla_client.py` is an SDK shell with default endpoint hardcoded `https://api.nixtla.io`; zero local model execution (deps: httpx/orjson/pydantic — no torch); forecasting/finetune/anomaly all server-side. Portables: async job submit→poll→cancel→fetch semantics (`jobs/_job.py`) for our remote-job design; `_audit.py` dataframe-ingestion-validation split for our D1 pipeline. The free Nixtla forecasters live in their OTHER repos (`statsforecast`, `mlforecast`, Apache-2.0, CPU-only) — that's where we'd go if we need forecast tooling	job-queue + audit patterns → §3/seat 4; never a fleet dependency
10	**timeplus-io/proton** (C++, Apache-2.0 ★2,276, commit 2026-10-28; 38,907 files — ClickHouse-engine streaming SQL; local read)	**ADOPT-CANDIDATE (tool trial at W2, after D1 harness baseline)**	biggest technical discovery of dossier D: **native NATS JetStream external stream** verified in code (`src/Storages/ExternalStream/NATSJetstream/*` registered in `ExternalStreamTypes.h`, smoke tests `0032_external_stream_nats/` incl. materialized-view case) — streaming materialized views (`SETTINGS recovery_policy='best_effort'`) could replace bespoke reducers for rolling per-symbol aggregates over our JetStream event log. NOT a D1 change: QuestDB stays D1 per §19; proton vies for the **D2 streaming-window layer**, run as a docker image	D2 candidate at W2 with its own gate: JetStream-speaking ingest parity + MV-recovery validation on our event shapes, head-to-head vs

#	Repo	Verdict	What we take	Where it slots
			(never rebuilt from source; ClickHouse-weight ops caveats + commercial-vendor awareness)	QuestDB-ILP windowing
11	**Open-Dev-Society/OpenStock** (TS, **AGPL-3.0 verified root LICENSE**, ★20,916, commit 2026-10-25; local read)	ADOPT-PATTERNS only (AGPL: no code lift into our private system; ideas porting only)	stealable *concepts* (re-implemented by us, not copied): market-session state machine incl. pre/open/after/closed + next-session countdown (`lib/market-session.ts`); quote-cache key-pool design with per-key in-flight caps (`lib/actions/finnhub.actions.ts`); step-function alert/email orchestration (`lib/inngest/functions.ts`); watchlist/alert schema shapes. CAVEATS: Finnhub 15-second-poll "realtime" (not tick feed); AI digest = plain LLM summarization; **no test/lint CI** (only docker-image workflow)	cockpit UI concepts; session-clock + alert-schema ideas onto Postgres/NATS
12	**mizorewww/laya-mlx** (Python, Apache-2.0, ★7,034)	**CONTRACT-ONLY** (runtime REJECT: MLX pinned darwin/arm64; CUDA explicitly unsupported — code read, `accelerate()` returns False / errors on CUDA). Take the typed-decision-head contract (choice / score / noul), abstention gates, calibration clamping (their own clamps shipped-warm bucket [0.5, 5.0] w/ RuntimeWarning) via upstream torch on Colab T4 if/when we want scoring heads	seat 7 decision-head design; no runtime dependency	

Net effect: two libraries adopted at code level as harness inputs (day-trader-skill skills + agency-agents personas, both already wired in this workspace), one small graph tool adopted (Graphify, §8.1), and four ported code segments (brigade loop- guards + approval contract + cron metering, ruflo swarm orchestrator contract) lifted into our own boss loop rather than running anyone's runtime wholesale — with NATS + Postgres + the L0–L3 enforcer as our own work regardless. No adopted code touches R1/R2. Three rejects are hard constraints: license-driven (AI-Trader no-license; OpenStock AGPL code boundary), hardware-driven (laya-mlx MLX), cost/cloud-driven (nixtla). Licenses are re-checked at intake time — repos can re-license.

§3.2 What none of the 13 give us (that we must build)

- Turnkey unattended fleet operation: brigade is the nearest (headless gateway daemon + cron scheduler + metering), but its transport/state layers are bespoke; the glue (NATS subjects ↔ Postgres tasks ↔ cockpit WS) is ours regardless.
- Highest-grade governance: task budgets, capability egress per agent, L0–L3 approval integration, D4 audit of every token spent per task.
- Fleet-specific vocabulary: our cells, our kill cascade, our D0–D7, our R1–R8.
- A cockpit that chats with the owner about *this* system.
- A scoring path that bridges LLM-produced scores → deterministic veto → D5 versioning.

§4 · Scoring / decision pipeline (seat 7 mechanics)

Pipeline (all offline; every stage deterministic given its inputs):

1. **Evidence in:** citable claims from seat 6 with quant-mind-style provenance (chunk index + page + verbatim quote + `as_of` timestamp). Un-cited statements are dropped by the scorer, not guessed.
2. **Typed scoring:** each candidate produces the laya-mlx *contract*: a `choice` distribution over labelled options, an expected `score` on a defined rubric, and a P(true). Implemented first as prompt-contract in a **deterministic harness** (day- trader-skill checklists are the grammar); laya-style tiny heads are an *optional* later train (Colab T4) — only if a calibration harness shows them beating the prompt-only baseline on held-out windows.
3. **Abstention/gates:** min_confidence / low_confidence flags like laya's abstention gates; scores that trip them are UNLABELED, not low-score versions.

4. **Calibration:** clamping like laya's (their shipped ckpt needed [0.5, 5.0]); any head/prompts get a wind-up period on past windows; measurement before belief.
5. **Versioned into D5:** score, inputs hash, prompt/model version, abstention flag, calibration status → new row; version that don't pass gates never link to live E2.
6. **E2 edge-lift gate:** an LLM-derived score is only trusted once it lifts measured edge on replay vs. the frozen baseline — same spirit as E4, replay-based, owner-read report. If the lift isn't there, we keep the old artifacts forever.

The upshot: **LLM decisions never leak into live trading unless they demonstrably pay for themselves in replay, repeatedly.**

§5 · Fence between LLM layer and trading fleet (§3+§4 rules, restated once)

- L0–L3 ladder applies from the control plane down to every shell command it runs.
- The LLM layer **reads** the same Postgres/NATS it cannot write to except its own tables (`llm_tasks` , cockpit notes, D5 pending scores). No D0/D1/D3-world writes except via approved W0a-harness commands.
- Every agent run gets a run_id; every failed run is investigated (flight-recorder replay), not paged through by a chat response.
- Chat transcriber worker archives every cockpit session to D4 + Obsidian vault.

§6 · Time-series store cross-ref (final, points at §19)

Full D0–D7 decision lives in `ENGINE-DATABASE-PLAN.md` §19 (Session 44): QuestDB D1, ClickHouse named fallback, TimescaleDB rejected for D1 (license wall: `tsl/` = Timescale License under the Apache-only build; CMake verified), Postgres stays D3/D4, Proton = streaming pattern-donor only. **The LLM fleet runs read-only against D1 for boot-prewarm and dashboards, and read/write against D3/D4/D5.** No engine changes from this session.

§7 · Skills architecture (sub-second precision, cheap defaults)

Each seat gets a **skill pack**, not a monolithic prompt. Adopted structure (mirrors trader-skill/agency-agents/quant-mind):

```
~/.opencode/fleet-skills/
  build-review/SKILL.md, audit-checklists.md (from agency-agents bodies)
  news-digest/SKILL.md, citation-contract.md (quant-mind artifact schema)
  trader-decision/SKILL.md + checklists (verbatim port of day-trader-skill grammar,
                   with our instruments and R9–R14 additions merged in)
  scoring-contract/SKILL.md (typed choice/score/noul + abstention + calibration)
  research-retrieval/SKILL.md (agentic walking of research store, cites required)
  cockpit-voice/SKILL.md (tone, honesty labels like ESTIMATE/MEASURED, output style)
```

Every skill file: YAML frontmatter, `description` carrying trigger phrases, ≤400 lines, a "do not" block (secrets, prod writes, un-logged actions). Loaded on demand (progressive disclosure like quant-mind's `contexts/`), so the base prompt stays small.

§8 · Cockpit layer — chat + observability

- Entry surface: an OpenCode/Claude session (" Cockpit ") that runs seat 12 with a read-mostly toolset and a view into D3/D4/D5.
- Per-topic reports the owner can ask for: cell health, position summaries, kill-state of each market, recent L0–L3 approvals needed, quality-of-LLM-layer report (how many scores passed E2 gates), budget burn, ticket burn-down.
- Everything the cockpit says is **labeled**: ESTIMATE/LABELLED-CLAIM/MEASURED — same discipline as this planning log.
- Chat streaming to D4; weekly digest note auto-appended to `gfi/` vault (Obsidian).

§8.1 All-system memory architecture (owner's Graphify/Obsidian ask)

The owner asked explicitly: *containers + Graphify + Obsidian for all system memory so agents recall faster and manage easier* — measure-then-adopt. Verdict:

Layer	What it is	What we already have	Adopt as	Why it pays	Risk to manage
Containers	Docker Compose wrapping every long-running process: Postgres (D3/D4), QuestDB (D1), NATS, cockpit UI, rail, backups, cockpit chat server	host processes, hand-run	**ADOPT** (W0c)	one-command cold start for agent drills and restart tests (W0a/W0b gates already require 10× restart); clean port/permission isolation; easy snapshot/rollback per service	resource slicing (8 GB box), socket systemd interplay, one more yaml to review — keep compose files in script-review
Obsidian	The vault is the **declarative decision memory**: decisions, session logs, research digests, runbook notes, promotion records; already home of this planning system	`gfi/` vault + `quant-nv` + `MANI-blueprints` (already registered as OpenCode references)	**FOLD IN** (W0c): formalize note types (decision / incident / runbook / research-digest) + mtime-indexed digests; cockpit weekly digest auto-writes via chat transcriber	notes already in our daily flow; zero new infra; cross-links surface related decisions	vault is our *working* docs — Graphify indexing must be read-only from it (no in-vault writes by repair bots)
Graphify	Graphify-Labs/graphify (`graphifyy`, Apache-2.0; source-read in W0c): local tree-sitter AST graph of the **codebase** + selected vault paths every time code/docs change (`--update` incremental). Written to `graphify-out/graph.json` + HTML; optional `--obsidian` note export for cross-linking into the vault	grep-based recall + ad-hoc context stuffing	**ADOPT as tool** (read-only from vault; do **not** adopt optional Cloud upsell — "80% token cut" is VENDOR-claimed)	AST cross-file `calls/imports/inherits` edges, EDGES labeled EXTRACTED vs INFERRED (honest trace); agents ask the graph before re-reading code; ~40 language grammars available; native opencode skill install (`~/.config/opencode/skills/graphify/`)	`--deep` /docs semantic pass uses Anthropic/DeepSeek APIs (extra egress + token spend → run code-only mode with `--no-viz` or without semantic backend by default); graph staleness if `--update` not run after edits: add as part of the build/review loop

Fit test (measured at W0c, not before) — primary trial run before making any of this load-bearing: run one representative cockpit question (e.g. "where does the NSE calendar live in code, and which tests cover D1 rests?") both with and without the Graphify graph, and one recall question against the vault with vs without Obsidian digest indexing. Adopt only whichever passes the *measured* context-token delta; put the numbers in the run log and update §8.1 otherwise.

§8.2 Memory per seat (per role add-on, not free-for-all)

Memory type	Store	Write rules
Episodic (what happened)	D3 `llm_tasks` + D4 audit (already planned)	agent writes own run record; nothing else
Semantic (what is known)	R3-approved D5 knowledge artifacts + Graphify graph.json (code) + vault notes (decisions) — recall substrate (brigade Tideline patterns vs ruflo `@claude-flow/memory`) decided by W0c toy-shootout §3	writes only via versioned artifact commit; no in-place edits
Procedural (how to do)	git code + skill files	PR + review; nothing silently patched
Working (current task)	task lease payload scope in Postgres	trimmed when task closed
Owner prefs / house rules	vault note types (§8.1)	cockpit seat 12 only, owner-readable diff

§9 · Research & coding harness hygiene

- Adopted repos land in `Trading & ai/llm-sources/` (or a sibling folder) with a `SOURCES` note listing license + commit-hash at intake (this is exactly the pattern that already governs `marketplaces/` here — no edits to vendored code).
- Any adopted vendored code is **read-outs + tests**, with attribution preserved; we do not edit adopting upstream code in place, per workspace guardrails.
- Claude Code and OpenCode, our two working harnesses, both point at the same skill files (§7) and the same Postgres task records — one source of truth, not two.

§10 · Security / privacy posture for the LLM layer

- **Model choices** (owner decision OPEN-14: model routing & privacy): primary model contract via current OpenCode/Claude; no PII beyond what owner feeds; no training on our code (only via vendor API terms); blacklist of models for cybersec/seat-3 work read from `opencode models` availability, not hard-coded.
- Secrets: never in prompts; `secrets vault` broker issues per-task short-lived tokens (W0c build item).
- Egress per seat (news: RSS/feeds; research: bookmarks + academic sites; cybersec: none). Violations → agent park + audit event.
- Hallucination controls: (a) citation-required artifacts (quant-mind contract); (b) numeric claims must quote a source artifact id; (c)written phrasing discipline (ESTIMATE/MEASURED/…) in cockpit; (d) benchmark gating before any score is trusted.

§11 · Self-learning loop (bounded, offline)

- Weekly: training seat re-runs E4-style *replay tests* on every score artifact still trusted; loses-list becomes tickets; research seat proposes candidate artifacts.
- Monthly: cross-cell merge review (which setups overlap? which duplicated?) — the same manual review loop as §8.5 of the fleet blueprint, kept human-descend liable.
- Hard stop: no LLM-written "learning" ever lands in R2 runtime without owner approval
  - its own E4/E2-gated promotion (never bypass this).

§12 · Gap register (everything Session 44 must fix or name)

#	Gap	Fix planned in	Notes
G1	Secrets vault does not exist yet	W0c	Xiting: hashicorp-vault or SOPS; pick in W0c run
G2	Observability for the LLM layer itself (token cost per agent/day, error rate)	W0c	simple Postgres counters + Grafana-lite; no LLM needed
G3	Backup/DR of cockpit + memory stores	W0c/§19	compose volumes + restore drills (already a W0a gate)
G4	Corporate-actions / calendar ingest for India + FX sessions	W2	O12-related data dependency
G5	LLM eval harness (ethical prompt-drift detector)	W0c	replay-question set + determinism check
G6	Reconciliation tooling (broker ↔ internal ledger)	W2/W3	seat 8, deterministic checks (not LLM)
G7	Local holiday/half-day calendars (NSE + FX)	W2	feeds G4
G8	Egress sandboxing (per-seat allow-lists)	W0c	enforce at orchestrator
G9	Cost caps dashboard for owner	W0c	simple SQL view

§13 · Build-order deltas (this blueprint folds into FULL-SYSTEM-PLAN Stage 6)

Wave	Added by this session
W0c (LLM control-plane skeleton)	brigade orchestrator stood up headless; `llm_tasks` table; L0–L3 plumbing; secrets vault G1; graphify + Obsidian fold-in (§8.1); §8.1 fit-test run; cost caps G9
W2	seats 4, 5, 11 runback; G4/G6/G7
W3–W4	seats 6, 7, 10 (research/scoring/fleet-control) with D5 + E2 gate; training seat rehearsals
W5	cockpit UI + chat plane GA; owner-only L3 flows; §8.1 re-measure

§14 · Open items raised + carried (registration book)

New:

- **O14** — model routing & privacy policy for seat selection per domain (vendor APIs vs local): *Owner question*: which models allowed for which seats; what data may leave the box (currently: everything source-only). *Needed by*: W0c.
- **O15** — NSE index membership & survivorship-bias-free vendor: must supply constituents-as-of-date + delisted names for PIT backtests. *Needed by*: W2 (before E4 historical runs on India).
- **O16** (from dossier A) — cockpit runtime choice: run **brigade's gateway daemon** (headless WS gateway + cron + approval bridge, but 44-day lull, WS-only, Convex/ pi-SDK baggage, brand-scrubbed provenance → strict vendor-pinned fork) **vs.** build a thin in-house cockpit on our stack (NATS + Postgres, riding the ruflo swarm contract + brigade loop-guard port). Both satisfy L0–L3; the inspection that decides is the W0c toy build *measuring*: cold-start time, restart drill fit for the 10× restart gate, and code volume we'd have to audit. Owner call; lean recommendation recorded at W0c with measurements.

Carried unchanged from S43: O3 (F&O cell granularity) · O4 (off-index names) · O5 (first style per instrument) · O6 (FX scalping add) · O7 (hedge-executor shape) · O8 (cTrader prod limits check) · O9 (Upstox algo-tagging) · O10 (order-balance rule) · O11 (HL FIU-IND/FEMA/tax) · O12 (PIT as-reported fundamentals) · O13 (ToS/ban avoidance checklist).

§15 · Sources

#	What	Class
S17	Session 44 sources: GitHub API metadata for all 13 owner repos (2026-10-28/29) · shallow clones at `/tmp/opencode/s44-repos/` (2026-10-29) with LICENSE-file reads · graphify source read (Apache-2, install.py opencode path, llm.py backend env) · `claude-code-memory-setup` README (lucasrosati; author-reported 71.5× claim, SEE ITS OWN CAVEAT) · Graphify-Labs/graphify README (VENDOR "80%" claim) · Context7/upstream-docs checks already logged in ENGINE-DATABASE-PLAN §19 (S16)	MIXED (labeled per row §3.1 / §8.1)